

Social Feed App



Projeto desenvolvido para vaga de Desenvolvedor Flutter Mobile em solicitação da empresa ESIG Group.

Um aplicativo de rede social moderno desenvolvido com **Flutter**, utilizando **Clean Architecture** e **Firebase** como backend. O app permite aos usuários criar, compartilhar e interagir com posts, incluindo funcionalidades de câmera, galeria e autenticação.



Tabela de Conteúdos

- [Visão Geral](#)
- [Arquitetura](#)
- [Requisitos](#)
- [Instalação](#)
- [Configuração](#)
- [Como Executar](#)
- [Estrutura do Projeto](#)
- [Dependências](#)
- [Troubleshooting](#)



Visão Geral

Social Feed App é uma aplicação completa de rede social que demonstra boas práticas em desenvolvimento Flutter, incluindo:

- ☒ Autenticação com Firebase Auth
- ☒ Armazenamento de dados em Firestore
- ☒ Upload de imagens no Firebase Storage
- ☒ Captura de fotos via câmera e galeria
- ☒ Gerenciamento de estado com MobX
- ☒ Injeção de dependência com GetIt
- ☒ Clean Architecture (Clean Code)
- ☒ Suporte multiplataforma (Android, iOS, Web)



Arquitetura

O projeto segue **Clean Architecture** dividido em 4 camadas principais:

1. **Core Layer** ([lib/core/](#))

Infraestrutura compartilhada da aplicação:

- **dependency_injection.dart**: Configuração do GetIt para injeção de dependências
- **firebase/**: Configuração inicial do Firebase

- **services/**: Serviços reutilizáveis (câmera, Firebase)
- **themes/**: Definição de temas e estilos globais

2. Data Layer (**lib/data/**)

Responsável por acessar dados (APIs, bancos de dados):

- **model/**: Modelos de dados para serialização/desserialização
 - **firebase_post_model.dart**: Modelo de post com dados do Firebase
 - **post_model.dart**: Modelo base de post
 - **user_model.dart**: Modelo de usuário
- **repository/**: Implementação dos repositórios
 - **firebase_post_repository.dart**: Acesso aos posts no Firestore
 - **services/**: Serviços de dados auxiliares
 - **utils/**: Utilitários de manipulação de dados

3. Domain Layer (**lib/domain/**)

Lógica de negócio pura, independente de implementação:

- **entities/**: Entidades do domínio (sem dependências externas)
 - **post.dart**: Entidade Post
 - **user.dart**: Entidade User

4. Presentation Layer (**lib/presentation/**)

Interface com o usuário e gerenciamento de estado:

Pages (**pages/**)

Principais telas da aplicação:

- **create_post/**: Tela de criação de posts
 - **create_post_page.dart**: Widget principal
 - **controllers/**: Controladores específicos
 - **widgets/**: Widgets reutilizáveis da página
- **feed/**: Tela de feed principal
 - **feed_page.dart**: Widget principal
 - **dialogs/**: Diálogos (confirmações, etc.)
 - **utils/**: Utilitários específicos
 - **widgets/**: Componentes da página
- **login/**: Tela de autenticação
 - **login_page.dart**: Widget principal
 - **widgets/**: Componentes de login
- **post_detail/**: Tela de detalhes do post

- `post_detail_page.dart`: Widget principal
- `dialogs/`: Diálogos
- `utils/`: Utilitários
- `widgets/`: Componentes

Stores (`stores/`)

Gerenciamento de estado com MobX:

- **`auth_store.dart`**: Lógica de autenticação
- **`auth_store.g.dart`**: Código gerado pelo MobX
- **`post_store/`**: Gerenciamento de posts
 - `post_store.dart`: Store principal
 - `computed/`: Propriedades computadas
 - `features/`: Funcionalidades específicas
 - `core/`: Utilitários centrais
 - `sync/`: Sincronização com Firebase
 - `utils/`: Helpers

Widgets (`widgets/`)

Componentes reutilizáveis:

- `comment_item.dart`: Item de comentário
- `custom_button.dart`: Botão customizado
- `image_preview.dart`: Prévia de imagens
- `universal_image.dart`: Carregador universal de imagens
- `post_card/`: Card de exibição de posts

Requisitos

Antes de começar, certifique-se de ter instalado:

- **Flutter SDK**: versão `^3.10.4`
 - [Instalar Flutter](#)
- **Dart SDK**: incluído no Flutter
- **Android SDK**: para desenvolvimento Android
 - Android 5.0 (API 21) ou superior
- **Xcode** (macOS): para desenvolvimento iOS
- **Visual Studio Code** ou **Android Studio**: editor recomendado

Verificar instalação:

```
flutter --version  
dart --version  
flutter doctor
```

Instalação

1. Clone o repositório

```
git clone https://github.com/seu-usuario/social_feed_app.git  
cd social_feed_app
```

2. Instale as dependências

```
flutter pub get
```

3. Gere código MobX (importante!)

```
flutter pub run build_runner build
```

Ou para modo watch (regenera automaticamente):

```
flutter pub run build_runner watch
```

Configuração

Configuração Firebase

O projeto utiliza Firebase para autenticação, Firestore e armazenamento. Siga os passos:

1. Criar projeto no Firebase Console

- Acesse [Firebase Console](#)
- Crie um novo projeto: `social-feed-app`

2. Configurar Android

```
flutter pub add firebase_core firebase_auth cloud_firestore firebase_storage
```

- No Firebase Console, adicione um app Android
- Download o `google-services.json`
- Coloque o arquivo em: `android/app/google-services.json`

3. Configurar iOS (opcional)

- No Firebase Console, adicione um app iOS
- Download o `GoogleService-Info.plist`
- Abra `ios/Runner.xcworkspace` no Xcode
- Arraste o arquivo para o projeto

4. Configurar Web (opcional)

O Firebase já está pré-configurado para web no `build/web/`

5. Habilitar serviços Firebase

No Firebase Console, habilite:

- **Authentication:** Ativar Google Sign-in
- **Firestore Database:** Criar banco de dados
- **Storage:** Criar bucket para imagens

6. Configurar regras de segurança Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Permite leitura e escrita apenas para usuários autenticados
    match /posts/{document=**} {
      allow read, write: if request.auth != null;
    }
    match /users/{document=**} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == resource.data.uid;
    }
  }
}
```

7. Configurar regras de segurança Storage

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /posts/{userId}/{allPaths=**} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == userId;
    }
  }
}
```

```
}  
}  
}
```

▶ Como Executar

Executar no emulador/dispositivo Android

```
flutter run
```

Executar em dispositivo específico

```
# Listar dispositivos disponíveis  
flutter devices  
  
# Executar em dispositivo específico  
flutter run -d <device_id>
```

Executar em modo debug

```
flutter run --debug
```

Executar em modo release

```
flutter run --release
```

Executar na Web

```
flutter run -d web-server
```

Depois acesse: <http://localhost:5000>

Executar testes

```
flutter test
```

📁 Estrutura do Projeto

```

lib/
├── main.dart                # Entrada da aplicação
├── core/                   # Camada de infraestrutura
│   ├── dependency_injection.dart
│   ├── firebase/
│   ├── services/
│   └── themes/
├── data/                   # Camada de dados
│   ├── model/
│   └── repository/
├── domain/                 # Camada de negócio
│   └── entities/
└── presentation/          # Camada de UI
    ├── pages/
    ├── stores/
    └── widgets/

```

Dependências

Principais

- **flutter_mobx**: Gerenciamento de estado reativo
- **mobx**: Base do gerenciamento de estado
- **get_it**: Injeção de dependência
- **firebase_core**: Core do Firebase
- **firebase_auth**: Autenticação
- **cloud_firestore**: Banco de dados
- **firebase_storage**: Armazenamento de arquivos
- **image_picker**: Captura de câmera/galeria
- **cached_network_image**: Carregamento e cache de imagens
- **photo_view**: Visualização de fotos
- **intl**: Internacionalização

Dev

- **build_runner**: Gerador de código
- **mobx_codegen**: Gerador de código MobX
- **flutter_lints**: Análise de código

Troubleshooting

Erro: "MobX generated files not found"

Solução:

```
flutter pub run build_runner build --delete-conflicting-outputs
```

Erro: "Firebase not initialized"

Solução: Certifique-se de que o `google-services.json` está em `android/app/`

Erro: "No devices found"

Solução:

```
# Iniciar emulador Android
emulator -avd <nome_emulador>

# Ou conectar dispositivo físico
adb devices
```

Erro: "Permission denied" ao acessar câmera

Solução: Verifique as permissões em `android/app/src/main/AndroidManifest.xml`

Hot Reload não funciona

Solução:

```
flutter clean
flutter pub get
flutter run
```

Build lento

Dica: Use modo release para testes de performance

```
flutter run --release
```

Fluxo de Autenticação

1. Usuário abre o app
2. Verifica se está autenticado (AuthStore)
3. Se não, vai para tela de Login
4. Realiza login com Firebase
5. Redirecionado para Feed principal
6. Pode criar posts, comentar e interagir

Funcionalidades Principais

Feed

- Exibir lista de posts
- Pull-to-refresh
- Loading infinito

Criar Post

- Selecionar imagem (câmera/galeria)
- Preview de imagem
- Escrever caption
- Upload automático

Detalhes do Post

- Ver comentários
- Adicionar comentários
- Like/Unlike
- Compartilhar

Perfil

- Visualizar informações do usuário
- Posts do usuário
- Logout

Boas Práticas

1. **Always run `flutter pub get`** depois de clonar o repositório
2. **Regenerate MobX code** sempre que modificar stores
3. **Use meaningful names** para variáveis e funções
4. **Test thoroughly** antes de fazer commit
5. **Keep layers separated** - não misture lógica em widgets
6. **Use `ChangeNotifier`** apenas quando necessário
7. **Optimize images** antes de adicionar ao assets

Recursos Úteis

- [Flutter Documentation](#)
- [Firebase Documentation](#)
- [Clean Architecture in Dart](#)
- [MobX Documentation](#)
- [GetIt Documentation](#)

Licença

Este projeto está sob licença MIT.

Autor

Sou desenvolvedor de aplicativos e iniciei minha trajetória na programação em 2022, trabalhando principalmente com Flutter e Dart. Atuei como desenvolvedor na Ponto Care entre 2022 e 2024, período em que tive a oportunidade de participar ativamente do desenvolvimento e evolução de aplicações reais, lidando com desafios do dia a dia de um produto em produção.

Desde o início, sempre busquei ampliar meu conhecimento explorando outras tecnologias e ferramentas, como Java para Backend, Node.js, Python, React Native, além de diversos serviços do Firebase. Também tive contato com plataformas como Coda e Notion, usando essas ferramentas para organização, automação e apoio a processos.



linkedin: LUIZ CARLOS